

Rapport de projet informatique

ARCADE LEXICON Site de jeux assisté par IA

Projet réalisé du 10 mars 2026 au 20 mai 2026

Membres du groupe

PARISOT Mathieu — N° étudiant : 45007467

CELIK Murat — N° étudiant : 45013521

BOUTROS Antonio — N° étudiant : 45014953

Code source : <https://github.com/45007467/Arcade-lexicon-projet-info>

Table des matières

1	Introduction	4
1.1	Contexte et objectifs	4
2	Environnement de travail	5
2.1	Outils de développement	5
2.2	Organisation du travail	5
3	Description du projet et objectifs	6
3.1	Présentation générale	6
3.2	Architecture du projet	6
3.3	Fonctionnalités principales	6
3.3.1	Système de compte utilisateur	6
3.3.2	Profil et statistiques	6
3.3.3	IA adaptative	7
3.3.4	Mémoire IA	7
4	Bibliothèques, outils et technologies	8
4.1	Langages utilisés	8
4.1.1	HTML5	8
4.1.2	CSS3	8
4.1.3	JavaScript ES6+	8
4.2	API Claude (Anthropic)	8
4.3	Algorithmes implémentés	9
5	Travail réalisé	10
5.1	Liste des fonctionnalités	10
5.1.1	Fonctionnalités réalisées	10
5.1.2	Fonctionnalités non réalisées	11
5.2	Répartition du travail	11
6	Difficultés rencontrées	12
6.1	Contrainte du fichier unique	12
6.2	Intégration de l'API Claude	12
6.3	Performance des algorithmes	12
6.4	Variété des parties	12
6.5	Mémoire IA personnalisée	12
7	Bilan	13
7.1	Conclusion	13
7.2	Perspectives	13
8	Bibliographie	14
9	Webographie	14
A	Architecture du code	15
B	Exemple d'exécution du projet	15

C	Manuel utilisateur	18
C.1	Prérequis	18
C.2	Lancement	18
C.3	Contrôles des jeux d'arcade	19

1 Introduction

Ce rapport présente le projet **ARCADE LEXICON**, un site web de jeux interactifs entièrement assisté par intelligence artificielle, réalisé dans le cadre du cours de programmation assistée par IA de Licence MIASHS première année à l'Université Paris Nanterre.

1.1 Contexte et objectifs

L'objectif principal de ce projet était de concevoir un site web de jeux intégrant une intelligence artificielle réelle, capable d'interagir avec le joueur de façon dynamique et personnalisée. Contrairement à un simple projet de programmation, ce projet met l'accent sur la **collaboration entre l'humain et l'IA** à toutes les étapes : conception, développement et expérience utilisateur finale.

Le projet répond aux exigences suivantes :

- Utilisation d'une IA générative (Claude d'Anthropic) intégrée directement dans le site
- Conception d'une expérience utilisateur originale et interactive
- Développement d'un système de compte utilisateur avec persistance des données
- Variété de jeux pour démontrer plusieurs usages de l'IA

2 Environnement de travail

2.1 Outils de développement

Le développement a été réalisé avec les outils suivants :

Outil	Usage
Claude (Anthropic)	Assistant IA principal pour la génération et l'aide au code
HTML5 / CSS3	Structure et mise en page du site
JavaScript (ES6+)	Logique des jeux, gestion des états, appels API
API Anthropic	Intégration de Claude dans le site pour les commentaires IA
LocalStorage (navigateur)	Persistance des comptes et statistiques utilisateurs
Canvas API	Rendu graphique des jeux d'arcade (Tetris, Snake, etc.)
Overleaf	Rédaction du rapport en $\text{\LaTeX}$
GitHub	Hébergement et versionnage du code source

TABLE 1 – Outils et technologies utilisés

2.2 Organisation du travail

Le projet a été développé de manière progressive et itérative sur plusieurs semaines, en utilisant Claude comme assistant de programmation principal. Plutôt que de suivre un plan figé dès le départ, nous avons avancé au fil des idées et des découvertes : chaque nouvelle fonctionnalité implémentée nous donnait de nouvelles idées que nous intégrions aussitôt au projet.

Concrètement, nous commençons par définir ensemble une fonctionnalité ou un jeu à ajouter, puis nous la décrivons à Claude qui génère le code correspondant. Nous testons ensuite le résultat, identifions les améliorations possibles ou les bugs, et recommençons le cycle. Cette méthode de travail agile nous a permis de partir d'un simple pendu pour arriver à un site complet avec 14 jeux, un système de compte, une IA adaptative et une mémoire des habitudes du joueur.

Ce mode de collaboration humain-IA a également été formateur : Claude expliquait systématiquement les concepts techniques utilisés (algorithme Minimax, API Canvas, LocalStorage, `async/await`), ce qui nous a permis de mieux comprendre le code produit et de prendre de meilleures décisions de conception au fil du projet.

3 Description du projet et objectifs

3.1 Présentation générale

ARCADE LEXICON est un site web de jeux rétro entièrement contenu dans un seul fichier HTML. Il se distingue par l'intégration profonde de l'intelligence artificielle à tous les niveaux du jeu.

L'interface adopte une esthétique *rétro terminal* (inspirée des écrans phosphore verts des années 80) avec des effets visuels CRT (scanlines, glow effect, flicker).

3.2 Architecture du projet

Le projet est constitué d'un **seul fichier HTML** (environ 160 000 caractères) qui contient :

- Toute la structure HTML (14 écrans de jeu)
- Tout le CSS (animations, effets visuels, thème rétro)
- Tout le JavaScript (logique des jeux, système de compte, appels IA)

Ce choix architectural garantit une portabilité maximale : le site fonctionne en ouvrant simplement le fichier dans un navigateur.

3.3 Fonctionnalités principales

3.3.1 Système de compte utilisateur

Un système complet d'authentification locale a été implémenté :

- Création de compte avec nom d'utilisateur et mot de passe (chiffré en base64)
- Connexion / déconnexion avec persistance via `localStorage`
- Plusieurs comptes possibles sur le même appareil
- Reconnexion automatique à la réouverture

3.3.2 Profil et statistiques

Chaque joueur dispose d'un profil détaillé comprenant :

- Score total, victoires, ratio VS IA, meilleure série
- Temps de jeu cumulé, parties jouées
- Tableau des statistiques par jeu (parties, victoires, meilleur score, taux de victoire)
- Graphique à barres du meilleur score par jeu
- Système de 13 badges/succès à débloquent
- Historique des 30 dernières parties
- Barre de niveau avec 6 paliers (Recrue -> Légende)

3.3.3 IA adaptative

L'IA s'adapte en temps réel aux performances du joueur :

- Niveau IA de 1 à 10, stocké dans le profil
- Augmente de +0,5 à chaque victoire, diminue de -0,3 à chaque défaite
- Modifie les prompts envoyés à Claude (mots plus difficiles, indices cryptiques, stratégie plus agressive)
- Visualisée dans le menu et dans le Tournoi

3.3.4 Mémoire IA

L'IA mémorise les habitudes du joueur et génère un rapport personnalisé :

- Lettres favorites au Pendu
- Colonne préférée au Puissance 4
- Style de jeu (agressif/défensif)
- Rapport IA avec faiblesse principale, point fort, prédiction, contre-stratégie

4 Bibliothèques, outils et technologies

4.1 Langages utilisés

4.1.1 HTML5

Structure de toutes les pages du site. Utilisation des éléments sémantiques, de `<canvas>` pour les jeux d'arcade, et des formulaires pour l'authentification.

4.1.2 CSS3

Mise en page, animations et effets visuels. Technologies CSS utilisées :

- Variables CSS pour le thème global
- Animations `@keyframes` (flicker CRT, pulse, bounce)
- `repeating-linear-gradient` pour l'effet scanlines
- Flexbox et Grid pour la mise en page responsive

4.1.3 JavaScript ES6+

Langage principal pour toute la logique. Fonctionnalités utilisées :

- Modules IIFE (*Immediately Invoked Function Expressions*) pour isoler chaque jeu
- `async/await` pour les appels à l'API Claude
- `localStorage` pour la persistance des données
- API Canvas 2D pour les graphismes des jeux arcade
- `requestAnimationFrame` pour les boucles de jeu

4.2 API Claude (Anthropic)

L'API Claude est au coeur du projet. Elle est appelée via `fetch()` en JavaScript :

Listing 1 – Exemple d'appel à l'API Claude

```
1 async function callClaude(prompt, maxTokens = 400) {
2   const response = await fetch(
3     'https://api.anthropic.com/v1/messages',
4     {
5       method: 'POST',
6       headers: { 'Content-Type': 'application/json' },
7       body: JSON.stringify({
8         model: 'claude-sonnet-4-20250514',
9         max_tokens: maxTokens,
10        messages: [{ role: 'user', content: prompt }]
11      })
12    }
13  );
14  const data = await response.json();
15  return data.content.map(b => b.text || '').join('').trim();
16 }
```

L'IA est utilisée pour :

- Générer les mots à deviner (Pendou, Wordle, Mot Mystère)
- Commenter les parties en temps réel
- Jouer le rôle de suspect (Le Procureur)
- Générer des définitions vraies/fausses (Duel Définitions)
- Analyser le profil du joueur (Mémoire IA)
- Animer le Tournoi adaptatif

4.3 Algorithmes implémentés

Algorithme	Usage
Minimax	IA du Morpion (imbattable) et du Puissance 4
Alpha-Beta pruning	Optimisation du Minimax pour le Puissance 4 (profondeur 4)
Chasse + ciblage	IA de la Bataille Navale (tire aléatoirement, cible les hits)
Fréquence lettres	IA du Pendu (joue les lettres les plus fréquentes en français)
RequestAnimationFrame	Boucles de jeu pour les 5 jeux d'arcade
LocalStorage JSON	Système de comptes et persistance des données

TABLE 2 – Algorithmes implémentés

5 Travail réalisé

5.1 Liste des fonctionnalités

5.1.1 Fonctionnalités réalisées

Fonctionnalité	État	Description
Le Pendu (solo + VS IA)	✓ Réalisé	Mode solo et duel simultané
Mot en 6 coups (Wordle)	✓ Réalisé	5/6/7 lettres, commentaires IA
Mot Mystère	✓ Réalisé	Indices progressifs, score décroissant
Duel Définitions	✓ Réalisé	L'IA invente de fausses définitions
Morpion VS IA	✓ Réalisé	Algorithme Minimax parfait
Puissance 4 VS IA	✓ Réalisé	Minimax + élagage alpha-bêta
Mastermind	✓ Réalisé	4 couleurs, 8 essais, feedback IA
Bataille Navale	✓ Réalisé	IA avec stratégie de ciblage
Le Procureur	✓ Réalisé	Interrogatoire d'un suspect IA
Détective	✓ Réalisé	Trouver l'anecdote mensongère
Tetris	✓ Réalisé	Rotation, ghost piece, niveaux
Snake	✓ Réalisé	4 modes de pommes différents
Space Invaders	✓ Réalisé	Vagues, tirs, comportement IA
Breakout	✓ Réalisé	5 patterns de briques générés aléatoirement
Frogger	✓ Réalisé	Voitures/rondins aléatoires par partie
Système de compte	✓ Réalisé	Login, register, localStorage
Profil + statistiques	✓ Réalisé	6 stats, graphiques, historique
13 badges/succès	✓ Réalisé	Débloqués automatiquement
IA adaptative (niveau 1-10)	✓ Réalisé	S'adapte à chaque victoire/-défaite
Mémoire IA	✓ Réalisé	Analyse personnalisée du joueur
Tournoi IA adaptatif	✓ Réalisé	5 manches avec difficulté croissante

TABLE 3 – Fonctionnalités réalisées

5.1.2 Fonctionnalités non réalisées

Fonctionnalité	Raison
Pac-Man	Trop complexe : labyrinthe + 4 IA de fantômes avec comportements différents (Chase, Scatter, Frightened). Nécessiterait un fichier séparé de plusieurs milliers de lignes.
Street Fighter / Mario Bros	Nécessite des sprites, une physique avancée et des animations image par image. Incompatible avec l'approche monofichier.
Vrai multijoueur en ligne	Nécessite un serveur backend (WebSocket). Non réalisable en HTML statique.
Hébergement en ligne	Nécessite une clé API Anthropic côté serveur pour sécuriser les appels. Possible avec Netlify Functions mais non mis en place.

TABLE 4 – Fonctionnalités non réalisées et justifications

5.2 Répartition du travail

Le projet a été réalisé à trois, en utilisant l'IA Claude comme outil d'assistance à la programmation. La répartition du travail s'est organisée comme suit :

Membre	Contributions principales
PARISOT Mathieu	Conception du projet, choix des jeux, coordination générale, tests et validation du site
CELIK Murat	Direction créative, choix du design rétro, spécifications des fonctionnalités IA, tests des jeux d'arcade
BOUTROS Antonio	Rédaction du rapport, mise en page $\text{\LaTeX}$, captures d'écran, documentation et manuel utilisateur

TABLE 5 – Répartition réelle du travail

6 Difficultés rencontrées

6.1 Contrainte du fichier unique

L’une des principales difficultés techniques a été de maintenir toute la complexité du projet (14 jeux, système de compte, appels API, graphismes Canvas) dans un seul fichier HTML. Cela a nécessité une organisation très rigoureuse du code JavaScript avec des modules IIFE pour éviter les conflits de variables.

6.2 Intégration de l’API Claude

Les appels à l’API Anthropic depuis un fichier HTML local posent des problèmes de sécurité (clé API exposée dans le code). En production, il faudrait un serveur intermédiaire (type Netlify Functions) pour masquer la clé. Pour ce projet scolaire, les appels sont faits directement depuis le navigateur dans l’environnement Claude.ai.

6.3 Performance des algorithmes

L’algorithme Minimax pour le Puissance 4 sans optimisation était trop lent (profondeur infinie). L’ajout de l’élagage Alpha-Bêta et la limitation à 4 coups de profondeur ont résolu ce problème tout en maintenant une IA très compétitive.

6.4 Variété des parties

L’exigence que “les parties ne se ressemblent pas” a nécessité plusieurs techniques :

- Génération aléatoire des mots par l’IA (Pendou, Wordle, Mystère)
- Patterns de briques différents selon le niveau (Breakout : 5 patterns : standard, diamant, croix, vague, bordure)
- Vitesses et directions aléatoires des obstacles (Frogger)
- 4 modes de pommes au Snake
- Commentaires IA toujours différents grâce à Claude

6.5 Mémoire IA personnalisée

La construction d’un contexte personnalisé pour chaque joueur et son injection dans les prompts Claude a demandé une réflexion sur la structure des données stockées et sur la façon d’extraire des informations pertinentes sans stocker trop de données.

7 Bilan

7.1 Conclusion

Ce projet a démontré qu'un étudiant débutant en programmation peut, avec l'aide d'une IA générative, concevoir et réaliser un site web complexe et original en quelques semaines. La collaboration avec Claude a permis d'atteindre un niveau de fonctionnalités (14 jeux, algorithmes Minimax, API IA intégrée, système de compte) qui aurait demandé des mois de développement traditionnel.

L'apport de l'IA ne s'est pas limité à la génération de code : elle a également joué un rôle de **conseillère pédagogique**, expliquant les concepts (Minimax, Canvas API, LocalStorage, async/await) au fur et à mesure de leur implémentation.

Du point de vue de l'expérience utilisateur, l'intégration de Claude dans les jeux eux-mêmes (commentaires en direct, adaptation de la difficulté, mémoire des habitudes) crée une expérience unique où chaque partie est différente et où l'IA semble véritablement "connaître" le joueur.

7.2 Perspectives

Les améliorations envisageables pour une version future sont :

- **Hébergement en ligne** avec sécurisation de la clé API (Netlify Functions)
- **Base de données réelle** (Firebase ou Supabase) pour un classement multi-joueurs global
- **Analyse d'image réelle** du dessin au Pendu via Claude Vision
- **Mode multijoueur** en temps réel (WebSocket)
- **Application mobile** (React Native ou PWA)
- **Pac-Man** avec implémentation des 4 algorithmes de fantômes originaux

8 Bibliographie

- [LAM94] L. LAMPORT, *TEX : A Document preparation system*, Addison-Wesley, 1994
- [ANT24] Anthropic, *Claude API Documentation*, Anthropic, 2024
- [GAR84] M. GARDNER, *Wheels, Life and Other Mathematical Amusements*, W.H. Freeman, 1983
- [KNU97] D. KNUTH, *The Art of Computer Programming Vol. 1*, Addison-Wesley, 1997

9 Webographie

- [ANT] <https://docs.anthropic.com> Documentation officielle de l'API Claude
- [MDN] <https://developer.mozilla.org> Documentation Canvas API et JavaScript
- [GIT] <https://github.com> Hébergement du code source
- [OVL] <https://overleaf.com> Compilation du rapport L^AT_EX

Annexes

Annexe A : Architecture du code

Le fichier principal `index.html` est organisé de la façon suivante :

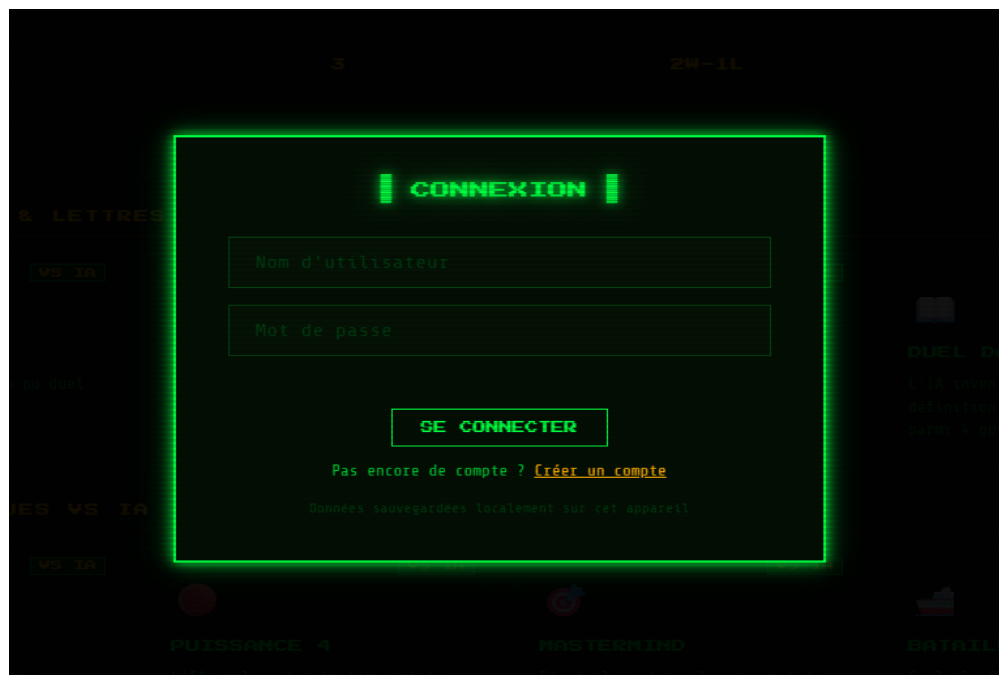
Listing 2 – Structure du fichier `index.html`

```
1 index.html
2 <style>           (CSS : theme retro , animations , styles des
   jeux)
3 <body>
4   Auth overlay ( ecran de connexion/inscription)
5   screen-menu  (menu principal avec 3 sections)
6   screen-profile (profil et statistiques)
7   screen-pendu / wordle / mystere / ... (14 ecrans de jeu)
8   screen-tetris / snake / invaders / breakout / frogger
9 <script>
10   DB {}           (gestion localStorage)
11   BADGES_DEF [] ( definition des 13 succes )
12   AUTH {} ( authentication )
13   IAmemory {} ( memoire et analyse IA)
14   ADAPT {}       (niveau IA adaptatif)
15   G.pendu {}     (jeu du Pendu)
16   G.wordle {}    (Wordle)
17   G.mystere {} (Mot Mystere )
18   G.morpion {}  (Morpion - Minimax)
19   G.p4 {} ( Puissance 4 - Alpha - Beta )
20   G.mastermind {} ( Mastermind )
21   G.bataille {} (Bataille Navale)
22   G.defduel {} (Duel Definitions )
23   G.detective {} ( Detective )
24   G.procureur {} (Le Procureur)
25   G.tetris {}   (Tetris - Canvas)
26   G.snake {}    (Snake - Canvas)
27   G.invaders {} (Space Invaders - Canvas)
28   G.breakout {} (Breakout - Canvas)
29   G.frogger {}  (Frogger - Canvas)
```

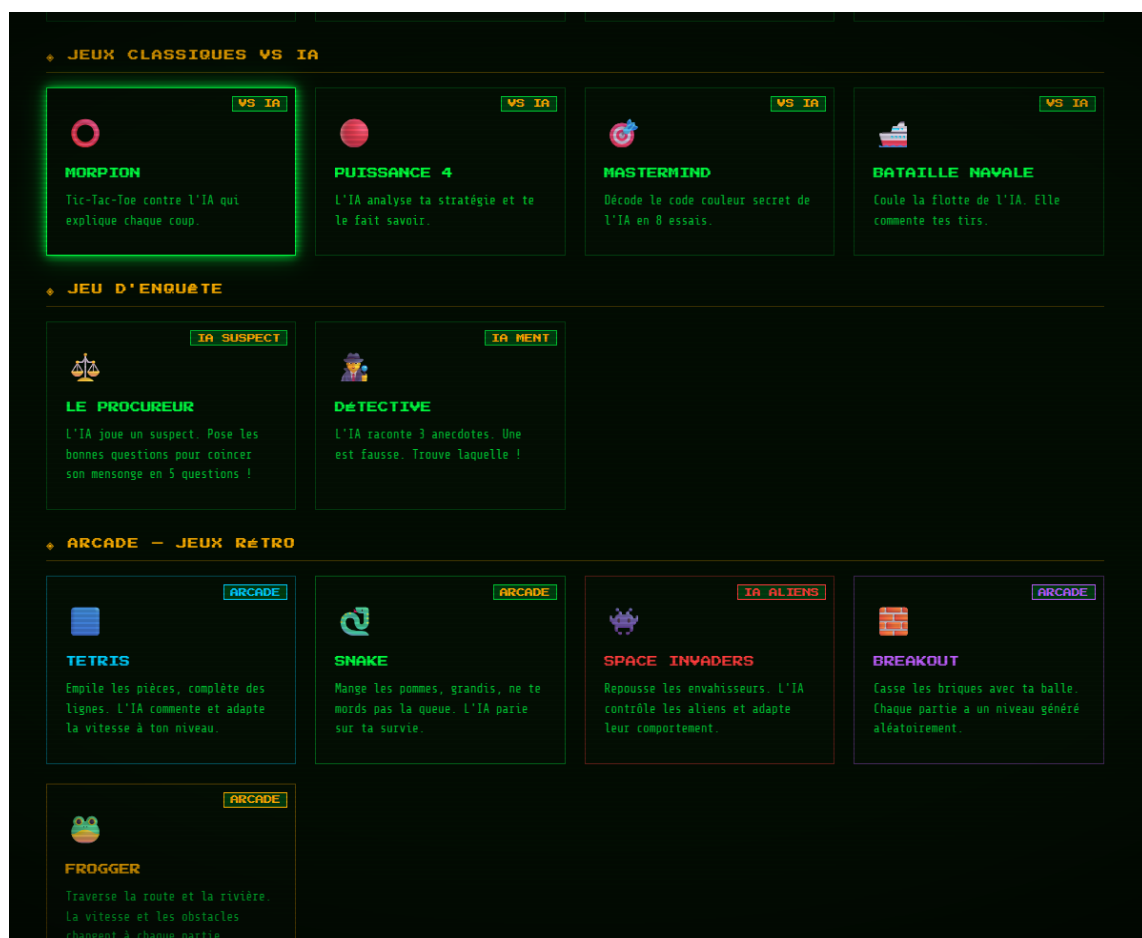
Annexe B : Exemple d'exécution du projet

Les captures d'écran suivantes illustrent les principales fonctionnalités du site :

1. Page de connexion : écran d'authentification avec création de compte



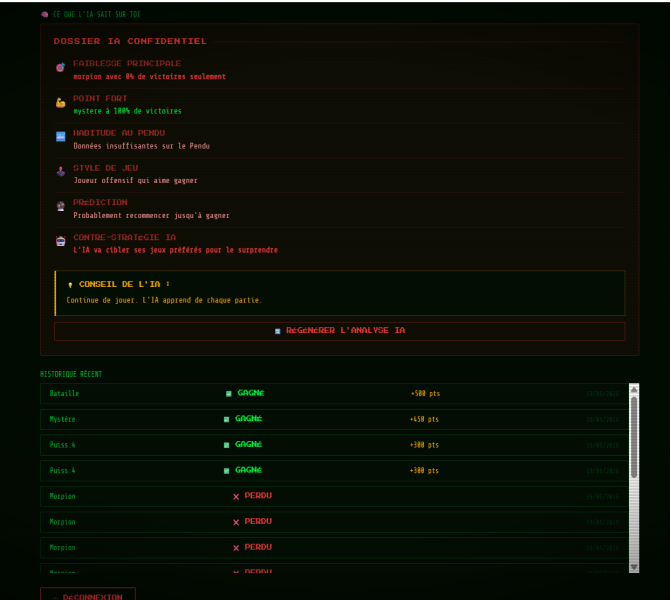
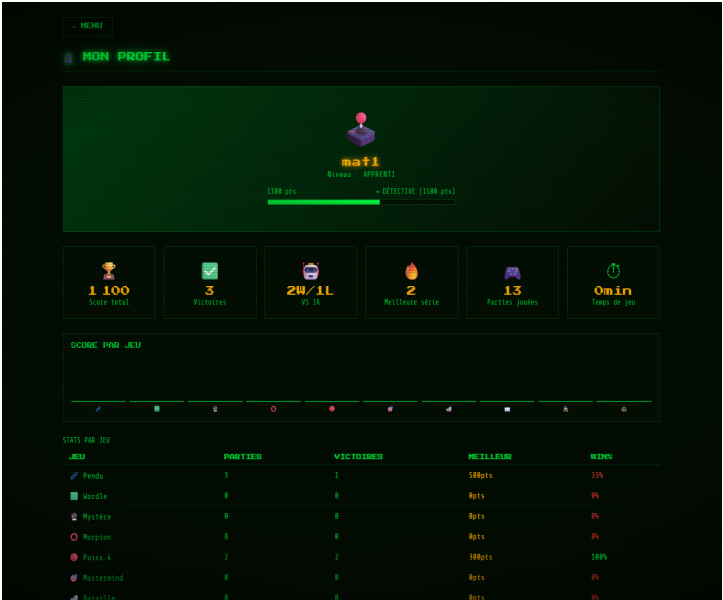
2. Menu principal : les 3 sections (Jeux de mots, Classiques VS IA, Arcade)



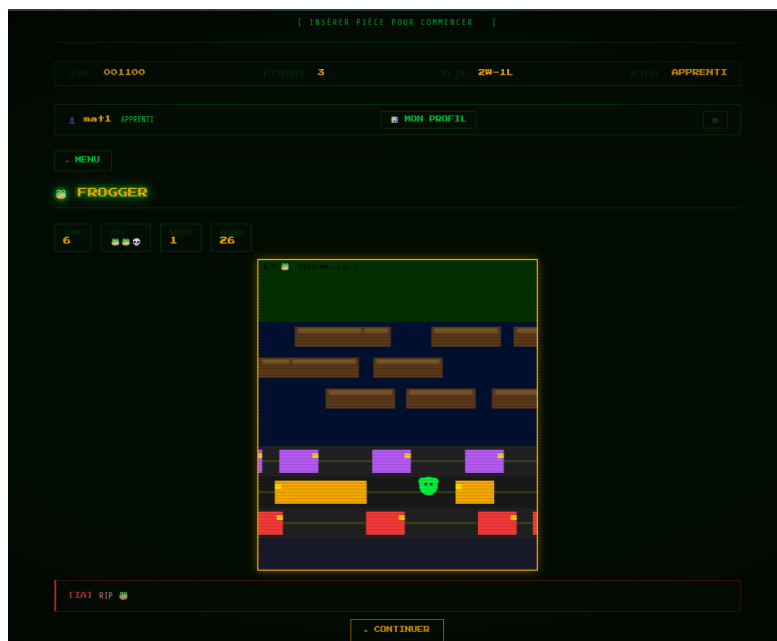
3. Le Pendu VS IA : duel simultané avec commentaires en temps réel



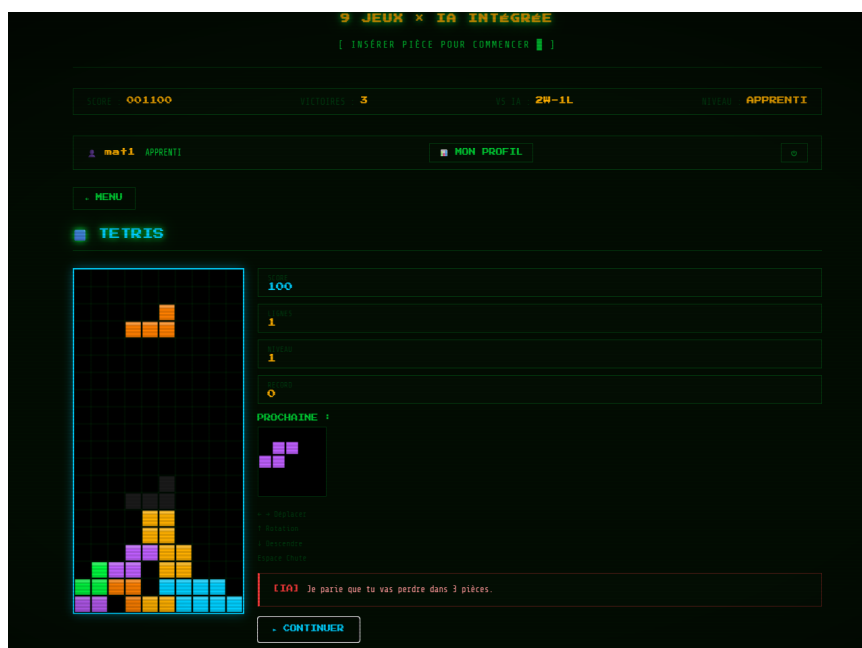
4. Profil utilisateur : statistiques, graphiques et badges



5. Frooger : le jeu de la grenouille



6. Tetris : jeu d'arcade avec ghost piece et niveaux



Annexe C : Manuel utilisateur

Annexe C.1 : Prérequis

Pour utiliser ARCADE LEXICON :

- Un navigateur web moderne (Chrome, Firefox, Edge, Safari)
- Une connexion internet (pour les appels à l'API Claude)
- Aucune installation requise

Annexe C.2 : Lancement

1. Télécharger le fichier `index.html`
2. Double-cliquer pour l'ouvrir dans votre navigateur
3. Créer un compte ou se connecter
4. Choisir un jeu dans le menu

Annexe C.3 : Contrôles des jeux d'arcade

Jeu	Contrôles
Tetris	<- -> Déplacer – ^ Rotation – v Descendre – Espace Chute rapide
Snake	Flèches directionnelles ou Z/Q/S/D
Space Invaders	<- -> Déplacer – Espace Tirer
Breakout	Souris ou <- ->
Frogger	Flèches directionnelles

TABLE 6 – Contrôles des jeux d'arcade